Retro OS Portfolio

Technical Specification & AI Implementation Guide v2.0

1. Vision & Core Architecture

A highly interactive, production-ready portfolio mimicking a classic desktop environment (Windows 98 / Classic Mac OS). The objective is to demonstrate advanced React state management, fluid animations, and a keen eye for UX design while remaining easily navigable for recruiters.

Desktop Layer

- Wallpaper/CRT background layer.
- Grid-based desktop icon system.
- Selection states (single click to highlight, double click to open).

Window Manager

- **Z-Index Tracking:** Clicking a background window brings it to the forefront.
- Draggable title bars and resizable edges.
- Controls: Minimize to taskbar, maximize to screen, close.

2. Refined Tech Stack

- **Framework:** React 19 (via Vite) - *A purely client-side SPA is ideal for a desktop simulation, avoiding SSR hydration mismatches with dynamic window coordinates.*
- **Styling:** Tailwind CSS v4 (with standard utility classes for rapid layout).
- **Window Mechanics:** `react-rnd` (Handles both drag and resize seamlessly, superior to react-draggable).
- **State Management:** `Zustand` - Essential for global window state (tracking open apps, coordinates, and active z-indexes without heavy prop drilling).
- **Animations:** `framer-motion` (For smooth window scaling/fading on mount/unmount).
- **Icons:** `lucide-react` or custom pixel-art SVG assets.

Critical Requirement: Mobile Strategy

Retro OS layouts fail on touch devices. The system must implement a strict CSS media query breakpoint (e.g., `max-width: 768px`).

- **Desktop:** Free-floating, overlapping, draggable `react-rnd` windows.
- **Mobile:** `react-rnd` is disabled. Icons turn into a standard list or grid grid. When opened, "windows" snap to 100% width/height of the viewport with a simple "Back/Close" button at the top, mimicking a mobile app rather than a desktop window.

3. State Management Schema (Zustand)

Your Zustand store should look similar to this to handle complex window logic:

```
interface WindowState {
  id: string;
  title: string;
  component: React.ReactNode;
  isOpen: boolean;
  isMinimized: boolean;
  isMaximized: boolean;
  zIndex: number;
}

// Actions needed in the store:
// - openApp(id, title, component)
// - closeApp(id)
// - minimizeApp(id)
// - maximizeApp(id)
// - focusApp(id) -> Increases highestZIndex and assigns to the target app
```

4. Phased AI Prompting Strategy (For VS Code / Agent)

Do not pass the entire project to the AI at once. Feed this document to your AI assistant to set context, then execute the build in these strict phases to ensure modularity and prevent spaghetti code.

Phase 1: Foundation & Desktop

```
"Phase 1: Scaffold a Vite + React project with Tailwind CSS v4 and Zustand.
Create the global layout: a Desktop container taking up 100vw/100vh with a
retro background (e.g., solid teal or retro pattern), and a bottom Taskbar
fixed to the bottom. Create a placeholder Start menu. Do not implement
windows yet."
```

Phase 2: Global State & Window Manager

```
"Phase 2: Implement the Zustand store for the Window Manager. The store
must track an array of open windows (id, title, isMinimized, zIndex) and
maintain a `highestZIndex` integer. Create an action `focusWindow(id)`
that increments `highestZIndex` and assigns it to the target window.
Create a dummy 'About Me' window to test the state."
```

Phase 3: Drag, Drop & Resize

```
"Phase 3: Integrate `react-rnd`. Map through the open windows in the
Zustand store and render them using `react-rnd` components on the Desktop.
Ensure the drag handle is restricted to the Window Title Bar. Bind the
onMouseDown event of the window wrapper to the `focusWindow` action so
clicked windows come to the front."
```

Phase 4: Content & Data Injection

```
"Phase 4: Create a dynamic content system. Define a JSON array of projects
(id, name, description, tech stack, github link). Create a Desktop Icon
component. When a Desktop Icon is double-clicked, it should dispatch an
action to the Window Manager to open a new window passing the project
details as props to a `ProjectViewer` component inside the window."
```

Phase 5: Animations & Polish

```
"Phase 5: Wrap the window components in Framer Motion ``.
Add a scale-up/fade-in effect when windows open, and scale-down/fade-out
when closed. Implement the Mobile Strategy: use Tailwind media queries
to disable dragging and force windows to `inset-0` (full screen) on
screens smaller than `md`."
```

5. Design & Aesthetic Guidelines

- **Typography:** 'MS Sans Serif' or 'Tahoma' for UI elements; 'IBM Plex Mono' for code blocks inside windows.
- **Borders:** Classic Windows 95/98 relies on "outset" and "inset" borders to create depth. Use pseudo-elements or specific box-shadows (e.g., `box-shadow: inset -1px -1px #0a0a0a, inset 1px 1px #fff, inset -2px -2px grey, inset 2px 2px #dfdfdf;`).
- **Performance:** If adding a CRT scanline or noise effect, do it via a lightweight CSS background image with `pointer-events: none` rather than heavy WebGL.